```
import numpy as np

class IrrigationACO:
    def __init__(self, num_zones, cost_matrix, num_ants=10, max_iter=100, alpha=1.0, beta=2.0, decay=0.5, Q=100):
        """
        Ant Colony Optimization customized for sequential agricultural irrigation scheduling.
        """
        self.num_zones = num_zones
        self.cost_matrix = cost_matrix
        self.num_ants = num_ants
        self.max_iter = max_iter
        self.alpha = alpha
        self.beta = beta
        self.decay = decay
        self.Q = Q

        # Initialize uniform global pheromones
        self.pheromones = np.ones((num_zones, num_zones))

        # Calculate heuristic visibility matrix (inverse of costs)
        adjusted_costs = np.copy(cost_matrix)
        np.fill_diagonal(adjusted_costs, np.inf)  # Prevent self-loop zero-division
        self.visibility = 1.0 / adjusted_costs

    def _pick_next_zone(self, current_zone, unvisited):
        """Selects the next field block using stochastic probability distribution."""
        unvisited_list = list(unvisited)

        # Compute numerators: (pheromone^alpha) * (visibility^beta)
        pheromone_factors = self.pheromones[current_zone, unvisited_list] ** self.alpha
        visibility_factors = self.visibility[current_zone, unvisited_list] ** self.beta
        attractiveness = pheromone_factors * visibility_factors

        total_weight = np.sum(attractiveness)
        if total_weight == 0:
            return np.random.choice(unvisited_list)

        probabilities = attractiveness / total_weight
        return np.random.choice(unvisited_list, p=probabilities)

    def optimize(self):
        best_route = None
        min_cost = float('inf')

        for iteration in range(self.max_iter):
            all_tours = []
            all_costs = []

            for ant in range(self.num_ants):
                start_zone = np.random.randint(self.num_zones)
                current_zone = start_zone
                unvisited = set(range(self.num_zones))
                unvisited.remove(current_zone)

                tour = [current_zone]
                total_cost = 0.0

                while unvisited:
                    next_zone = self._pick_next_zone(current_zone, unvisited)
                    total_cost += self.cost_matrix[current_zone, next_zone]
                    tour.append(next_zone)
                    unvisited.remove(next_zone)
                    current_zone = next_zone

                # Complete the tour by returning to the starting zone
                total_cost += self.cost_matrix[current_zone, start_zone]
                tour.append(start_zone)

                all_tours.append(tour)
                all_costs.append(total_cost)

                # Track global elite path
                if total_cost < min_cost:
                    min_cost = total_cost
                    best_route = tour
```

```
            # Step: Pheromone Evaporation
            self.pheromones *= (1.0 - self.decay)

            # Step: Pheromone Deposition (Ant-Cycle model)
            for tour, cost in zip(all_tours, all_costs):
                deposit_amount = self.Q / cost
                for i in range(self.num_zones):
                    from_node = tour[i]
                    to_node = tour[i+1]
                    self.pheromones[from_node, to_node] += deposit_amount
                    self.pheromones[to_node, from_node] += deposit_amount

        return [int(zone) for zone in best_route], min_cost
# --- Operational Execution Block ---
if __name__ == "__main__":
    # Simulating 5 distinct field quadrants needing water.
    # Elements capture transition weight (e.g., pipe surge lag + peak-hour electricity costs).
    farm_cost_matrix = np.array([
    [0, 10, 15, 20, 19],
    [10, 0, 35, 25, 11],
    [15, 35, 0, 30, 14],
    [20, 25, 30, 0, 6],
    [19, 11, 14, 6, 0]], dtype=float)

    # Initialize the framework
    scheduler = IrrigationACO(
        num_zones=5,
        cost_matrix=farm_cost_matrix,
        num_ants=8,
        max_iter=40,
        alpha=1.0,
        beta=2.0,
        decay=0.4
    )

    optimal_sequence, minimal_cost = scheduler.optimize()

    print(f"Optimal Field Irrigation Sequence: {optimal_sequence}")
    print(f"Minimized System Operational Cost Index: {minimal_cost}")
```

```
Optimal Field Irrigation Sequence: [1, 0, 2, 4, 3, 1]
Minimized System Operational Cost Index: 70.0
```